

CS 4375: Introduction to Machine Learning

Project 3: Deep Learning for MNIST and CIFAR-10

In this project, you will implement and evaluate Multilayer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs) on two well-known image classification datasets: **MNIST** and **CIFAR-10**. You will utilize **PyTorch** for implementation. The MNIST and CIFAR-10 datasets are readily available through the `torchvision.datasets` library in PyTorch.

For help with PyTorch, refer to the official documentation at: <https://pytorch.org/docs>.

1 Datasets and Preprocessing

- Use PyTorch (`torchvision`) to load MNIST and CIFAR-10 datasets.
- Normalize pixel values to $[0,1]$ or use standard normalization.
- Clearly document your preprocessing steps.

2 Part 1: Multilayer Perceptrons (MLPs)

- Implement MLPs that accept flattened images from MNIST and CIFAR-10.
- Evaluate **at least three distinct architectures**:
 1. Shallow (1 hidden layer, e.g., 128 units)
 2. Medium-depth (3 hidden layers, e.g., [512, 256, 128])
 3. Deep (at least 5 hidden layers, your choice)
- For each architecture and dataset:
 - **Validation-based hyperparameter tuning** using a held-out validation set (e.g., 45k/5k for CIFAR-10; 50k/10k for MNIST).
 - Tune:
 - * Learning rate (LR) {0.01, 0.001, 0.0001}
 - * Batch size {32, 64, 128}
 - * Optimizer (SGD vs Adam)
 - * Dropout {0.2, 0.5}
 - Instead of testing all 36 combinations, explore **10–20 meaningful configurations**.
 - Select best model using validation accuracy, retrain on full training+validation data, and report test accuracy.
 - Present results using Tables 1 and 2.

Table 1: MNIST Results (MLPs)

Architecture	LR	Batch	Opt	Dropout	Test Acc	Runtime
MLP (shallow)	...	...	...	...	...	...
MLP (medium)	...	...	...	...	...	...
MLP (deep)	...	...	...	...	...	...

Table 2: CIFAR-10 Results (MLPs)

Architecture	LR	Batch	Opt	Dropout	Test Acc	Runtime
MLP (shallow)	...	...	...	...	...	...
MLP (medium)	...	...	...	...	...	...
MLP (deep)	...	...	...	...	...	...

3 Part 2: Convolutional Neural Networks (CNNs)

- Evaluate **two CNN architectures**:
 1. **Simple CNN**: 2 convolutional layers (e.g., 8 filters each), with batch normalization and pooling, followed by a fully connected layer.
 2. **Enhanced CNN**: at least 3 convolutional layers with increasing filters (e.g., $16 \rightarrow 32 \rightarrow 64$), batch normalization, and pooling after each layer.
- Use:
 - 3×3 filters
 - Batch normalization after each convolution
 - Adam optimizer (fixed)
 - Stride = 1
- Tune:
 - Learning rate $\{0.01, 0.001, 0.0001\}$
 - Batch size $\{32, 64, 128\}$
 - Weight decay (L2 regularization) (e.g., $\{0, 10^{-4}, 5 \times 10^{-3}\}$)
- Explore **10–20 meaningful configurations** per architecture.
- Select best model using validation accuracy, retrain, and report results using Tables 3 and 4.
- Discuss impact of:
 - depth
 - number of filters
 - downsampling strategy

Table 3: MNIST Results (CNNs)

Architecture	LR	Batch	Weight Decay	Test Acc	Runtime
CNN (simple)	...	...	...	...	...
CNN (enhanced)	...	...	...	...	...

Table 4: CIFAR-10 Results (CNNs)

Architecture	LR	Batch	Weight Decay	Test Acc	Runtime
CNN (simple)	...	...	...	...	...
CNN (enhanced)	...	...	...	...	...

4 Experimental Guidance (Important)

- For all hidden layers in both MLPs and CNNs, use **ReLU** activations. The final output layer should produce raw logits. Do not apply a softmax activation explicitly, as `CrossEntropyLoss` in PyTorch internally applies softmax during training.
- **Validation splits must be clearly documented.** Clearly state how you partitioned training vs. validation data (e.g., 45k/5k for CIFAR-10, 50k/10k for MNIST).
- **Carefully manage runtime.** On CPU, expect approximate training times per model:
 - MNIST: MLP ~5–10 minutes, CNN ~10–20 minutes
 - CIFAR-10: MLP ~15–25 minutes, CNN ~30–60 minutes
- **GPU (Colab recommended)** significantly reduces runtime (MNIST: seconds, CIFAR-10: 1–3 minutes per training run).
- **Early stopping:** You may stop training when validation accuracy plateaus or begins to decrease. Report the stopping epoch if you use early stopping.

4.1 Strong Recommendation: Use Google Colab (Free GPU)

To efficiently complete this deep learning project, we **strongly recommend** using **Google Colab**, which provides **free GPU access**. Colab significantly speeds up training (10–20x faster than CPU), making your experimentation and hyperparameter tuning quicker and easier.

- Access Colab: <https://colab.research.google.com/>
- GPU Setup: In Colab, click on: Runtime → Change runtime type → Hardware accelerator: GPU (T4 GPU)

5 Grading Criteria

- Correct implementation and clear use of validation splits: 40 points
- Hyperparameter exploration and reporting: 30 points
- Clear, detailed analysis of results (tables, comparisons, and **justification of final model**): 20 points

- Code clarity and reproducibility: 10 points

6 What to turn in

Submit a single zip file containing:

- A PDF report (6-8 pages; **tables may extend beyond the page limit**) clearly describing:
 - Detailed architectures tested
 - Hyperparameter tables with validation results (accuracy $\pm$ std)
 - Test-set results for your final chosen models
 - Discussion explaining why certain hyperparameters/architectures performed better on a dataset, supported by results from your tables
 - Describe one key challenge you faced and how you resolved it.
- Your AI chat transcript (in the same format as Project 1).
- Your code with a README file with instructions for running it.

Your code must compile and reproduce your results exactly, or no credit will be given. To ensure reproducibility, try setting random seeds in your code and mention it in your report.